

Sockets in Python

Redes e Serviços

Daniel Corujo, Paulo Salvador, Miguel Matos

23 April 2026

Exercises

Practical Lab: Sockets in Python

Objective: Run the provided socket scripts, observe the behaviour of UDP and TCP connections at the packet level with Wireshark, and understand the trade-offs between threading, I/O multiplexing, and binary protocol encoding.

Use the class VM, which already has Python installed and configured, and run the provided Python scripts within its localhost.

Open a Wireshark capture on the **loopback interface** (lo) for all exercises. A useful display filter to isolate traffic on port 5005:

```
tcp.port == 5005 or udp.port == 5005
```

Part 1: UDP Sockets

Not all network communication requires a connection. The two most common transport protocols, **UDP** and **TCP**, represent two fundamentally different models.

UDP (User Datagram Protocol) is *connectionless*. A sender simply calls `sendto()` with a destination address and the data is transmitted as a self-contained datagram. There is no handshake, no state maintained between sender and receiver, and no delivery guarantee. If a datagram is lost, reordered, or duplicated, neither side is notified. UDP is appropriate when low latency matters more than reliability. DNS queries, live video, and real-time games all use UDP.

TCP (Transmission Control Protocol) is *connection-oriented*. Before any data is exchanged, a three-way handshake must complete:

```
Client          Server
|--- SYN ----->|
|<--- SYN-ACK ----|
|--- ACK ----->|
|                 |
|=== data =====|
```

Only after this handshake does either side have a socket it can use to read and write. TCP guarantees that bytes arrive in order and without loss. The OS acknowledges every segment and retransmits anything that is dropped. This reliability comes at a cost: latency for setup and overhead for acknowledgement tracking.

In Python, the distinction maps directly to the `socket()` call:

Protocol	Type constant	Characteristic call
UDP	<code>socket.SOCK_DGRAM</code>	<code>sendto()</code> / <code>recvfrom()</code>
TCP	<code>socket.SOCK_STREAM</code>	<code>connect()</code> / <code>send()</code> / <code>recv()</code>

1.1 Starting the Server

Open a terminal on the class VM and start the UDP server:

```
$ python3 serverUDP.py
```

The server creates a `SOCK_DGRAM` socket, binds to `0.0.0.0:5005`, and waits. Unlike TCP, there is no `listen()` or `accept()`, since UDP has no concept of a connection. The server calls `recvfrom()`, which blocks until a datagram arrives and returns both the data and the sender's address.

1.2 Sending Messages

Open a second terminal and start the client:

```
$ python3 clientUDP.py
```

Send a few messages. In Wireshark, observe each exchange: a single UDP datagram goes from client to server, and nothing comes back; the server only prints and does not reply.

Note the source port the client uses. Stop the client (`Ctrl+C`) and restart it. Check the source port again.

Thinking Question: The client's source port changes every time the process starts. Why does the OS assign a different ephemeral port on each run? If two instances of `clientUDP.py` were running simultaneously, how would the server tell them apart?

Thinking Question: Start the client *before* the server. Send a message. Does the client block, error, or silently continue? Compare this behaviour to TCP. What does this reveal about what UDP guarantees?

Part 2: TCP Sockets

When a TCP server calls `accept()`, it gets back a new socket connected to one specific client. The server must now decide: how does it serve this client while also remaining available to accept new ones?

The simplest answer is **one thread per client**. After `accept()`, the server spawns a new thread and hands it the client socket. The main loop returns immediately to `accept()` and waits for the next connection. Each thread runs its own blocking `recv()` loop independently.

```
while True:
    client_sock, address = server.accept()
    t = threading.Thread(target=handle_client, args=(client_sock, address), daemon=True)
    t.start()
```

Part 2A: TCP with Threads

2A.1 Starting the Server

Start the threaded TCP echo server:

```
$ python3 serverTCP.py
```

The server creates a `SOCK_STREAM` socket, binds, and calls `listen(5)`. The argument in `listen(arg)` is the *backlog*, the maximum number of pending connections the OS will queue before refusing new ones. The main loop calls `accept()`, which blocks until a client connects and returns a new socket dedicated to that client. A daemon thread is spawned for each accepted connection.

2A.2 Connecting a Client

With Wireshark running, open a second terminal and connect:

```
$ python3 clientTCP.py
```

Send a few messages. Observe in Wireshark:

1. The three-way handshake (SYN → SYN-ACK → ACK) before any data appears.
2. Each message and its echo as separate TCP segments (or combined, depending on timing).
3. The FIN/ACK teardown when the client terminates.

Note that the client's source port stays constant for the entire session; unlike UDP, a TCP connection is a stateful pairing of source IP, source port, destination IP, and destination port.

2A.3 Multiple Clients

Open two additional terminals, each running `clientTCP.py`. Send messages from all three simultaneously.

Confirm in the server output that each connection is logged with its own address and handled independently.

Thinking Question: The server spawns a new OS thread for each accepted connection. What happens to memory and scheduling overhead as the number of simultaneous clients grows toward thousands? At what point does this model become impractical?

Thinking Question: The `recv(1024)` call in `handle_client_connection` reads up to 1024 bytes. If a client sends a message longer than 1024 bytes, or sends two messages very quickly, what does the server actually receive in a single `recv` call? What does this tell you about relying on `recv` to return exactly one message?

Part 2B: TCP with Selectors

2B.1 Starting the Server

Start the selector-based server:

```
$ python3 serverTCPsel.py
```

Inspect `serverTCPsel.py`. The server socket is set to non-blocking (`setblocking(False)`) and registered with `selectors.DefaultSelector`. The event loop calls `sel.select()`, which blocks until at least one registered socket is ready. The registered callback for the server socket accepts the new client and registers *that* socket too. The registered callback for a client socket reads and echoes data.

Both the server socket and all client sockets share a single thread, and the selector dispatches to the right callback based on which socket became ready.

2B.2 Connecting a Client

Run `clientTCP.py` against this server (same client, same port):

```
$ python3 clientTCP.py
```

Send messages. The behaviour is identical to Part 2A from the client's perspective.

Thinking Question: The selector server uses one thread; the threaded server uses one thread per client. From the client's perspective, is there any observable difference in the responses? Where exactly does the difference between the two models manifest?

Thinking Question: Consider a low-end embedded device or microcontroller running a simple TCP service, with very limited RAM and no OS-level thread scheduler. Which model — threaded or selector-based — is more appropriate, and why? What is the main cost that makes the other model impractical? What about for higher-end devices, such as datacenters with huge amounts of CPU/cores?

Part 3: Packet Sizes

Text-based protocols encode data as human-readable strings: the number 42 is sent as the two bytes "42". This is convenient for debugging but wasteful and ambiguous: what separates one field from the next? What is the maximum value? Text forces you to parse.

Binary protocols encode data in fixed, typed fields. The number 42 as a 4-byte unsigned integer is always exactly `\x00\x00\x00\x2a`, regardless of its decimal representation. A receiver that knows the format can unpack it without parsing.

Python's `struct` module converts between Python values and C-style packed binary data. The key functions are:

Function	Description
<code>struct.pack(fmt, v1, v2, ...)</code>	Pack values into a bytes object
<code>struct.unpack(fmt, buffer)</code>	Unpack a bytes object into a tuple of values
<code>struct.calcsize(fmt)</code>	Return the byte size of the packed format

Format strings combine a byte-order prefix with type characters:

Prefix	Meaning
<code>!</code>	Network byte order (big-endian). Always use this for network protocols.
<code>=</code>	Native byte order. Not portable across architectures.

Character	C type	Python type	Size
<code>B</code>	unsigned char	<code>int</code>	1 byte
<code>L</code>	unsigned long	<code>int</code>	4 bytes
<code>s</code>	<code>char[]</code>	<code>bytes</code>	1 byte per char

A format string like `'!BLL20s'` describes: network byte order, one unsigned byte, two unsigned longs, and a 20-byte string, totalling 29 bytes. Once both sides agree on a format string, no parsing is needed: `unpack` returns the values directly.

Two strategies exist for variable-length payloads:

- **Fixed-size packets:** pad every payload to the same length. Simple to implement: the receiver always reads the same number of bytes. Wastes bandwidth for short messages; messages longer than the fixed size must be split by the sender, adding overhead on the client side.
- **Variable-size packets:** send a fixed header that includes the payload length, then send the payload. The receiver reads the header first, extracts the length, then reads exactly that many bytes. This is the same strategy as the 2-byte length prefix you implemented in the previous class.

Part 3A: Fixed Packet Sizes

3A.1 Packet Structure

The v2 scripts use a fixed-size binary packet. Every packet is exactly **29 bytes**:

Offset	0	1	2	3	4	5	6	7	8
	+-----+-----+-----+-----+-----+-----+-----+-----+								
	ver	order (4 bytes)					size (4 bytes)		
	+-----+-----+-----+-----+-----+-----+-----+-----+								
Offset	9	10	...					28	
	+-----+-----+-----+-----+-----+-----+-----+-----+								
	message (20 bytes, padded with '-')								
	+-----+-----+-----+-----+-----+-----+-----+-----+								

Python `struct` format string: `'!BLL20s'`

Field	Type	Size	Description
ver	B (unsigned char)	1 byte	Protocol version, always 1
order	L (unsigned long)	4 bytes	Sequence number, increments per message
size	L (unsigned long)	4 bytes	Actual message length before padding
message	20s	20 bytes	Message text, right-padded with ' - '

Note that the message field is capped at 20 bytes. If the user enters more than 20 characters, the client truncates the input silently; the excess bytes are discarded. To send longer messages, the sender would need to split them across multiple packets.

Start the server:

```
$ python3 serverTCPv2.py
```

3A.2 Sending Binary Packets

Start the client:

```
$ python3 clientTCPv2.py
```

Send a short message (e.g., hello). In Wireshark, find the TCP segment carrying the payload. Switch to the raw bytes view and locate:

- Byte 0: `\x01` (version)
- Bytes 1–4: sequence number in big-endian
- Bytes 5–8: original message length in big-endian
- Bytes 9–28: message text padded with ' - '

Thinking Question: The size field encodes the original message length, but the payload is always padded to 20 bytes. How does the receiver recover the original message? What would happen if the size field were absent? Could the receiver still strip the padding?

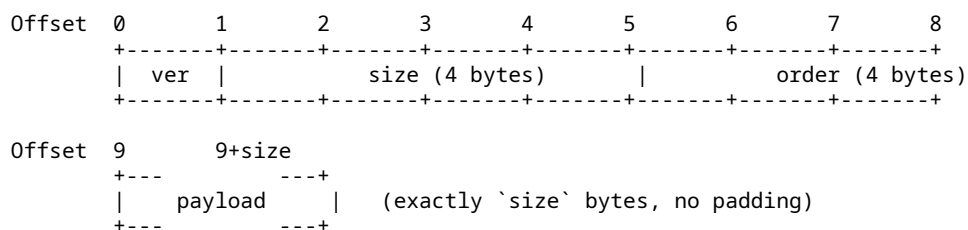
3A.3 Coding Exercise

`serverTCPv2.py` currently only receives and prints. It sends no reply. Extend it to echo back the same 29-byte packet to the client. Extend `clientTCPv2.py` to receive the echo and unpack it using the same format string.

Part 3B: Variable Packet Sizes

3B.1 Packet Structure

The v3 scripts use a two-part packet: a **fixed 9-byte header** followed by a **variable-length payload**:



Header format string: `'!BLL'` (9 bytes)

Field	Type	Size	Description
ver	B	1 byte	Protocol version, always 1

Field	Type	Size	Description
size	L	4 bytes	Exact byte length of the payload that follows
order	L	4 bytes	Sequence number
payload	variable	size bytes	Message text, no padding

Note that the field order differs from v2 (size comes before order here). These are two independent protocol definitions.

Start the server:

```
$ python3 serverTCPv3.py
```

3B.2 Observing Variable-Length Payloads

Start the client:

```
$ python3 clientTCPv3.py
```

Send messages of different lengths. In Wireshark, observe that the TCP payload length changes with each message: a 3-character message produces a 12-byte payload (9-byte header + 3 bytes), while a 10-character message produces a 19-byte payload.

Compare this to Part 3A, where every TCP payload was exactly 29 bytes regardless of message length.

The server reads in two steps:

```
# Step 1: read the fixed header
request = client_socket.recv(struct.calcsize(headerformat)) # always 9 bytes
version, size, order = struct.unpack(headerformat, request)

# Step 2: read exactly `size` bytes of payload
request = client_socket.recv(size)
```

Thinking Question: Why does the server call `recv()` twice instead of once with a large buffer? Suppose the client sends a 9-byte header followed immediately by a 50-byte payload. Is it guaranteed that the first `recv(9)` returns exactly 9 bytes and nothing more?

Thinking Question: Compare v2 (fixed-size, 29 bytes always) and v3 (variable-size, header + payload). What are the trade-offs between the two approaches? Under what circumstances would fixed-size packets be preferable despite wasting bandwidth on padding?

3B.3 Coding Exercise

`serverTCPv3.py` currently only receives and prints. Extend it to echo back the payload to the client using the same header+payload format. Extend `clientTCPv3.py` to receive and print the echo.